Project 2 Report Q1-Q10

Akshara, Lydia, and Mandar

Implementation Note: This solution uses unrestricted Damerau-Levenshtein distance rather than Optimal String Alignment (OSA) to allow overlapping transpositions, which is required to achieve the expected minimum cost for the test cases.

1 Breaking Down the Problem into Subproblems

To solve this problem, I use an extended DP formulation. Specifically, $d[i+1][j+1]$ represents the minimum cost to transform the first i characters of the target string T into the first j characters of the typo string S.

The key insight is that each state $d[i][j]$ depends on:

- **Match or substitute:** $d[i-1][j-1]$ plus substitution cost (0 if matching)
- **Delete:** $d[i-1][j]$ plus deletion cost
- **Insert:** $d[i][j-1]$ plus insertion cost
- **Transpose:** For characters that were adjacent in the original but may have been separated by other operations, we jump back to $d[i_1][j_1]$ (the last positions where these characters appeared) and add costs for any intermediate operations plus the transposition itself

This formulation allows transpositions to overlap, meaning a character can participate in multiple transpositions, which the simpler OSA variant does not permit.

The answer is $d[n+1][m+1]$, built from base cases upward.

2 Parameters of the Recursive Function

In the iterative implementation, we use indices i and j where:

- i ranges from 1 to n (number of characters processed from target)
- j ranges from 1 to m (number of characters processed from typo)
- $d[i+1][j+1]$ stores the minimum cost to transform the first i characters of T into the first j characters of S

3 Recurrence

The recurrence for unrestricted Damerau-Levenshtein is:

For each i from 1 to n and j from 1 to m:

- Let $i_1 = DA[S[j]]$ (last row where $S[j]$ appeared in T)
- Let $j_1 = DB$ (last column where $T[i]$ matched in S)

$$d[i+1][j+1] = \min \begin{cases} d[i][j] + \text{substitutionCost}(T[i], S[j]) & [0 \text{ if } T[i] = S[j]] \\ d[i][j+1] + \text{deletionCost}(i) & \\ d[i+1][j] + \text{insertionCost}(i, j) & \\ d[i_1][j_1] + \text{delBetween} + \text{tCost} + \text{insBetween} & \text{(if valid)} \end{cases}$$

where for the transposition case:

- delBetween = sum of deletion costs from i_1 to $i-2$
- insBetween = sum of insertion costs from j_1 to $j-2$
- tCost = transpositionCost$(T[i_1 - 1], T[i-1])$
- Valid when $i_1 > 0$, $j_1 > 0$, $T[i_1 - 1] = S[j]$, $T[i-1] = S[j_1]$

Base cases:

- $d[0][0] = \infty$ (guard value)
- $d[i+1][1] = \sum_{k=0}^{i-1} \text{deletionCost}(k)$ for $i \geq 0$
- $d[1][j+1] = \sum_{k=0}^{j-1} \text{insertionCost}(k)$ for $j \geq 0$
- All $d[i][0]$ and $d[0][j] = \infty$ (guards)

4 Base Cases of the Recurrence

The base cases establish boundary conditions:

Base case 1: Empty strings. $d[1][1] = 0$ when both strings start empty (after accounting for guard row/column at index 0).

Base case 2: Target finished, typo remains. $d[1][j+1]$ equals the cumulative cost of inserting the first j characters of the typo string.

Base case 3: Typo finished, target remains. $d[i+1][1]$ equals the cumulative cost of deleting the first i characters of the target string.

These base cases ensure the DP can handle any combination of string lengths.

5 Data Structure for Recognizing Repeated Subproblems

The repeated subproblems are identified by pairs of indices (i, j). The data structures needed are:

Primary DP table:

- $d[n+2][m+2]$: stores minimum cost for each state
- Size $(n+2) \times (m+2)$ to accommodate guard rows/columns at indices 0

Transposition tracking (for unrestricted DL):

- DA: hash map (character $\rightarrow$ integer)
  - $DA[c]$ = last row index where character c appeared in target
  - Updated after processing each row
- DB: integer variable
  - Tracks the last column where current target character matched
  - Reset for each new row, updated when characters match

Optional (for reconstruction):

- prev$[n+2][m+2]$: stores backpointers to reconstruct the operation sequence

This implementation is efficient: DP table lookups are $O(1)$, and DA lookups are $O(1)$ average case with hash map.

6 Pseudocode for Iterative Dynamic Programming Algorithm

Algorithm 1 SolveTypo(target, typo)

Input: target string of length n, typo string of length m
Output: minimum cost
Initialize:
$d \leftarrow$ 2D array $(n+2) \times (m+2)$, filled with ∞
$DA \leftarrow$ hash map, initialize all characters to 0
$d[0][0] \leftarrow \infty$
for $i = 0$ to n **do**
    $d[i+1][0] \leftarrow \infty$
    $d[i+1][1] \leftarrow \sum_{k=0}^{i-1} \text{deletionCost}(k)$
end for
for $j = 0$ to m **do**
    $d[0][j+1] \leftarrow \infty$
    $d[1][j+1] \leftarrow \sum_{k=0}^{j-1} \text{insertionCost}(k)$
end for
Main DP:
for $i = 1$ to n **do**
    $DB \leftarrow 0$
    for $j = 1$ to m **do**
        $i_1 \leftarrow DA[\text{typo}[j-1]]$
        $j_1 \leftarrow DB$
        if $\text{target}[i-1] = \text{typo}[j-1]$ **then**
            cost $\leftarrow 0$; $DB \leftarrow j$
        else
            cost $\leftarrow \text{substitutionCost}(\text{target}[i-1], \text{typo}[j-1])$
        end if
        $c_1 \leftarrow d[i][j] + \text{cost}$
        $c_2 \leftarrow d[i][j+1] + \text{deletionCost}(i-1)$
        $c_3 \leftarrow d[i+1][j] + \text{insertionCost}(i-1, j-1)$
        $c_4 \leftarrow \infty$
        if $i_1 > 0$ and $j_1 > 0$ and $\text{target}[i_1-1] = \text{typo}[j-1]$ and $\text{target}[i-1] = \text{typo}[j_1-1]$ **then**
            $\text{delBetween} \leftarrow \sum_{k=i_1}^{i-2} \text{deletionCost}(k)$
            $\text{insBetween} \leftarrow \sum_{k=j_1}^{j-2} \text{insertionCost}(k)$
            $\text{tCost} \leftarrow \text{transpositionCost}(\text{target}[i_1-1], \text{target}[i-1])$
            $c_4 \leftarrow d[i_1][j_1] + \text{delBetween} + \text{tCost} + \text{insBetween}$
        end if
        $d[i+1][j+1] \leftarrow \min(c_1, c_2, c_3, c_4)$
    end for
    $DA[\text{target}[i-1]] \leftarrow i$
end for
return $d[n+1][m+1]$

7 Worst-Case Time Complexity

The worst-case time complexity is $O(n \cdot m)$, where n is the length of the target string and m is the length of the typo string.

For each of the $n \times m$ cells in the DP table, we:

- Compute 4 candidate costs (match/substitute, delete, insert, transpose) in $O(1)$ each
- For transpositions, compute bridge costs which in the worst case iterate over $O(\max(i - i_1, j - j_1))$ elements, but amortized across all cells this remains $O(n \cdot m)$
- Update DA and DB in $O(1)$

The DA hash map operations (lookup and update) are $O(1)$ average case.
Total: $O(n \cdot m)$ time complexity.

8 Nested Loops for an Iterative DP Solution

The iterative solution uses nested loops to fill the DP table bottom-up:

Outer loop: for $i = 1$ to n (iterate over target characters)
Inner loop: for $j = 1$ to m (iterate over typo characters)
Compute $d[i+1][j+1]$ based on:

- $d[i][j]$ (match or substitute)
- $d[i][j+1]$ (delete from target)
- $d[i+1][j]$ (insert from typo)
- $d[i_1][j_1]$ (transpose, where i_1 and j_1 are tracked via DA/DB)

The key difference from simple edit distance: the transpose case doesn't just look at $d[i-1][j-1]$ or $d[i-2][j-2]$, but can reference any earlier cell $d[i_1][j_1]$ based on character positions tracked in DA and DB.

9 Space Complexity Improvement

The standard implementation requires $O(n \cdot m)$ space for the DP table.

For simple edit distance, space can be reduced to $O(\min(n, m))$ by keeping only the current and previous row. However, for unrestricted Damerau-Levenshtein, this optimization is not straightforward because:

- The transposition case can reference arbitrary previous cells $d[i_1][j_1]$, not just adjacent rows
- We need access to potentially any earlier row depending on character positions

Therefore, the full $O(n \cdot m)$ table is generally required for this implementation.

Alternative: If transpositions are rare or bounded, specialized space-saving techniques could be used, but these add significant complexity for modest space savings.

10 Advantages and Disadvantages of the Iterative Algorithm

Advantages of the iterative true Damerau-Levenshtein algorithm:

- Handles overlapping transpositions correctly, achieving optimal cost
- No recursion overhead or stack depth limits
- More predictable memory usage (no hidden recursion stack)
- Better cache locality from sequential table access
- Easier to add optimizations like early termination

Disadvantages:

- More complex to implement than simple OSA due to DA/DB tracking
- Requires full $O(n \cdot m)$ space (cannot easily optimize to $O(\min(n, m))$)
- Reconstruction requires explicit backtracking through the table
- Bridge cost calculations in transpositions add implementation complexity

Overall, the iterative true DL approach is necessary for this problem because the expected optimal solutions require overlapping transpositions that OSA cannot handle.